OmniSupport AI — Final AI Engineering Capstone

Industry-style integrated assessment across Sessions 1–30 | Revised student brief

Assessment at a glance

Student role	Primary data	Supporting assets	Production evidence
AI Engineer on a retail support modernisation team	10 Parquet shards × 100,000 = exactly 1,000,000 records	480 images; 40 policies/SOPs; 10,000-row fine-tuning subset; 15,000-order agent store	API/UI, Docker, CI, automated tests, logs and observable monitoring

Assessment purpose. Demonstrate whether you can independently connect the skills from Sessions 1–30 into one coherent, workplace-style AI engineering solution. This is not 30 separate exercises and not a tutorial clone.

1. Business scenario

OmniSupport is a large online retailer handling customer-support requests through web chat, email, mobile and phone. Management wants an AI-assisted support platform that can analyse historical operations, predict outcomes, understand ticket text and return images, retrieve company policy, answer grounded questions, and execute controlled local support tools while preserving human approval for sensitive actions.

Your task is to build one integrated prototype that connects analytical modelling, deep learning, vision, NLP/transformers, LLM engineering, RAG, agent tool use, evaluation and basic production packaging.

2. Stakeholders and business objectives

- **Support operations:** identify delay/escalation patterns and prioritise cases.
- **Customer-service leaders:** understand drivers of resolution time and segment recurring support demand.
- **Support agents:** retrieve relevant policy evidence and use controlled tools safely.
- **Returns team:** classify synthetic return/damage images and support return/refund decisions.
- **Engineering:** expose selected capabilities through a reliable local API/UI with tests, containerisation, CI, logging and measurable monitoring.

3. Available data and assets

- **Primary analytical dataset:** `data/raw/support_records_part_001.parquet` to `_010.parquet`, exactly 1,000,000 records in total.
- **Preview:** `dataset_preview.csv` for quick schema inspection only.
- **Data-quality challenges:** controlled missing/blank values, duplicates, outliers, category inconsistencies, class imbalance, noisy text, subgroup imbalance and post-outcome leakage fields.
- **Vision:** 480 synthetic labelled return images across damaged, packaging-damage, wrong-item and normal classes.
- **Transformer subset:** 10,000 ticket text/label rows for mandatory small-model fine-tuning.
- **Agent store:** local orders/customers/returns CSVs for safe tool-calling; do not perform real external actions.
- **RAG knowledge base:** 40 Markdown policy/SOP documents with overlapping and specific rules.

- **Evaluation:** prompt, RAG, agent and monitoring requirements plus starter test contracts.

4. Mandatory project phases

Phase	Required outcome
1 — Business understanding & framing	Define users, decisions, targets, prediction timing, success metrics, assumptions and risks.
2 — Data audit, cleaning & EDA	Profile all primary shards; address missingness, duplicates, outliers, category inconsistencies, imbalance and leakage; communicate findings with useful visuals.
3 — Feature engineering	Create leakage-safe numerical, categorical, date/time, interaction and text-derived features and justify them.
4 — Classical ML	Complete at least one regression and one classification task; compare baselines/models; use suitable metrics, cross-validation, tuning and threshold analysis.
5 — Unsupervised learning	Produce and validate a defensible customer/ticket segmentation; interpret cautiously.
6 — Deep learning	Train a small tabular neural network with train/validation loops, loss/optimiser experiments, reproducibility and checkpoint evidence.
7 — Computer vision	Train a basic CNN and compare it with transfer learning; include augmentation, confusion matrix and error analysis.
8 — NLP, attention & transformers	Build BoW/TF-IDF text classification and semantic search; complete the mandatory attention task and mandatory transformer fine-tuning task.
9 — Prompting & LLM application engineering	Build reusable prompts and Pydantic-validated structured outputs; benchmark at least 20 LLM requests for latency and token/cost evidence.
10 — RAG	Chunk policies, embed/index, retrieve evidence, generate grounded answers with sources, compare chunking settings and test no-answer cases.
11 — Agent & tool calling	Implement local tools, schemas, controlled orchestration, missing-information handling, failures and human approval gates.
12 — Evaluation, guardrails & responsible AI	Test quality/failures, hallucination, privacy, subgroup/fairness risk, content safety and auditability; create a risk register.
13 — API/UI integration	Expose selected features through FastAPI and/or Streamlit/Gradio with request validation and controlled errors.
14 — Docker, CI/CD, logging & monitoring	Containerise, keep CI active, extend automated tests, collect logs, and produce the required monitoring summary.
15 — Final report & demo	Provide architecture, reproducibility, evaluation, deployment/CI, monitoring, limitations, risks and trade-off reasoning.

5. Four explicit compulsory session checks

Session 20 — Attention mechanism

Using one customer-support message, explain **query, key, value, self-attention and positional information**. Include either a worked numerical example or a visualisation. Then explain how the representation differs from TF-IDF. A generic definition-only paragraph is insufficient.

Session 22 — Hugging Face fine-tuning

Perform a real small-model text-classification fine-tuning/adaptation using `data/subsets/transformer_finetune_10000.parquet`. Evaluate on held-out data and compare with your TF-IDF baseline. If hardware requires a smaller training sample, document the limitation but still complete an actual training run.

Session 25 — LLM app performance

Run at least **20 representative LLM requests**. Record latency and estimated input/output token usage/cost. Explain at least one trade-off involving quality, latency, cost, context-window use, streaming, API-key handling or UX.

Session 30 — CI and observable monitoring

Keep `.github/workflows/tests.yml` operational and extend tests for your implemented components. Produce a monitoring artifact containing request count, average latency, failures, RAG no-answer rate, agent tool failures, model version and prompt version.

6. Agent safety and test-case expectations

- Use only synthetic/local project data for tools such as order lookup, customer lookup, proposed refund calculation, policy search and return eligibility.
- A02 uses `ORD00001002`, which exists in the supplied agent store.
- A04 intentionally provides no order ID. The correct behaviour is to request the missing identifier **before** calling order-related tools.
- Refunds above frontline authority and other sensitive/state-changing actions must stop for human approval; the prototype must not perform real external actions.
- Tool failures/not-found cases must be surfaced safely rather than answered with invented data.

7. Evaluation expectations

- Choose metrics that match the target, imbalance and business consequence; do not rely on accuracy alone.
- Use honest train/validation/test logic and prevent duplicate leakage and preprocessing leakage.
- For RAG, evaluate retrieval/source correctness as well as generated answers and abstention behaviour.
- For LLM prompts/agents, include failure cases, schema validation and repeatable regression tests.
- For responsible AI, address privacy, bias/subgroups, hallucination, unsafe actions, auditability and appropriate human oversight.

8. Required deliverables

- Readable repository with source/notebooks, pinned environment, README and architecture explanation.
- Evidence for every phase and the four compulsory session checks above.
- Evaluation tables/figures, model/RAG/agent failure analysis and responsible-AI risk register.

- FastAPI/UI demonstration, Docker instructions, CI evidence and monitoring output.
- Short final demo/report explaining engineering decisions and trade-offs.

9. Marking overview

Area	Points
Problem framing	5
Data quality	6
EDA & features	5
Classical ML	8
Evaluation & tuning	6
Clustering	4
Deep learning	6
Vision	6
NLP/attention/transformers	9
Prompting/LLM performance	7
RAG	7
Agent/tools	7
Evaluation/guardrails/testing	6
Responsible AI	5
API/UI	4
Deployment/CI/monitoring	5
Code/docs/reasoning	4
Total	**100**

10. Constraints and integrity

- Stay within Sessions 1–30. Advanced infrastructure or post-course techniques are not compulsory.
- Do not use instructor-only generation/hidden-evaluation files.
- Do not hard-code answers to supplied evaluation cases.
- Do not expose API keys/secrets or perform irreversible external actions.
- You must be able to explain submitted code and decisions in the viva.

The assessment rewards valid engineering process, evaluation and reasoning. Suspiciously high metrics produced by leakage or invalid testing may receive less credit than lower but honest, well-explained results.